



```
In [6]: import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import SGD
import matplotlib.pyplot as plt
from tensorflow.keras.callbacks import Callback
```

```
In [7]: # Input data
X = np.array([[0,0], [0,1], [1,0], [1,1]])
# Target data
y = np.array([[0], [1], [1], [0]])
```

```
In [8]: def create_model(lr=0.1, momentum=0.0):
    model = Sequential()

    model.add(Dense(8, input_dim=2, activation='relu'))
    model.add(Dense(1, activation='sigmoid'))

    optimizer = SGD(learning_rate=lr, momentum=momentum)

    model.compile(optimizer=optimizer, loss='mse')

    return model
```

```
In [12]: class LearningRateLogger(Callback):
    def __init__(self):
        self.lrs = []

    def on_epoch_end(self, epoch, logs=None):
        lr = float(self.model.optimizer.learning_rate)
        self.lrs.append(lr)
```

```
In [13]: losses = []
learning_rates = []

converged = False
target_loss = 0.001
epochs_done = 0
lr = 0.1

model = create_model(lr)

lr_callback = LearningRateLogger()

while not converged:

    history = model.fit(
        X, y,
        epochs=10,
        verbose=0,
        callbacks=[lr_callback]
```

```

)

loss = history.history['loss'][-1]

losses.append(loss)
learning_rates.append(float(model.optimizer.learning_rate))

epochs_done += 10

if loss < target_loss:
    converged = True

```

```

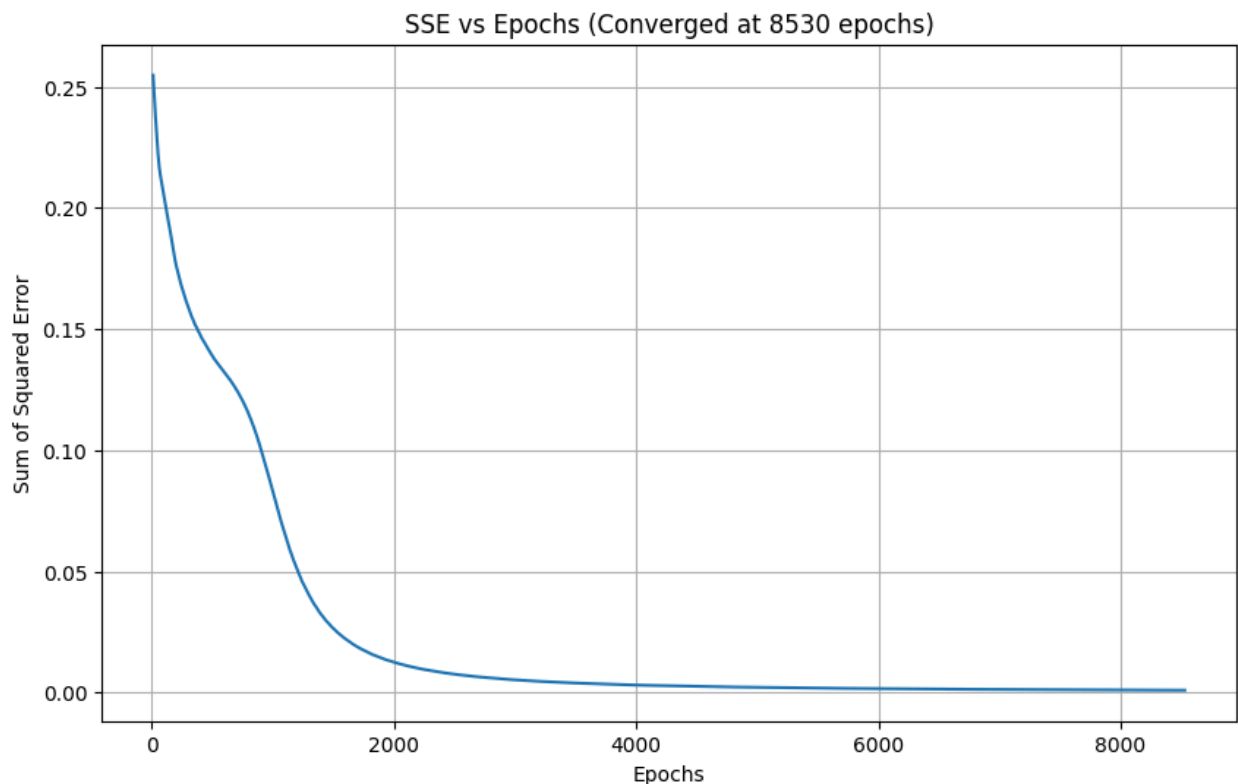
In [15]: plt.figure(figsize=(10,6))

plt.plot(np.arange(10, epochs_done + 1, 10), losses)

plt.title(f"SSE vs Epochs (Converged at {epochs_done} epochs)")
plt.xlabel("Epochs")
plt.ylabel("Sum of Squared Error")

plt.grid(True)
plt.show()

```



```

In [16]: plt.figure(figsize=(10,6))

plt.plot(np.arange(10, epochs_done + 1, 10), learning_rates)

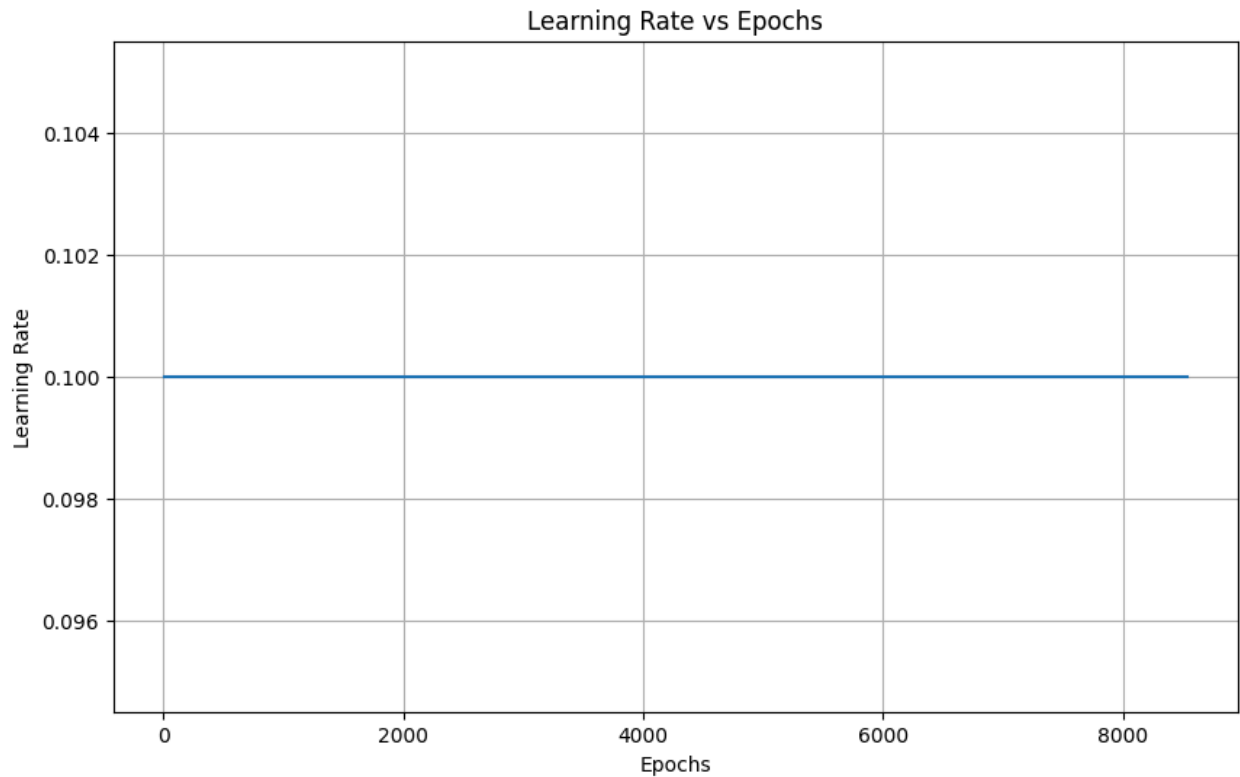
plt.title("Learning Rate vs Epochs")
plt.xlabel("Epochs")

```

```
plt.ylabel("Learning Rate")
```

```
plt.grid(True)
```

```
plt.show()
```



```
In [17]: losses_mom = []
learning_rates_mom = []

converged = False
lr = 0.1
momentum = 0.9
epochs_done = 0

model = create_model(lr, momentum)

lr_callback = LearningRateLogger()

while not converged:

    history = model.fit(
        X, y,
        epochs=10,
        verbose=0,
        callbacks=[lr_callback]
    )

    loss = history.history['loss'][-1]

    losses_mom.append(loss)
```

```

learning_rates_mom.append(float(model.optimizer.learning_rate))

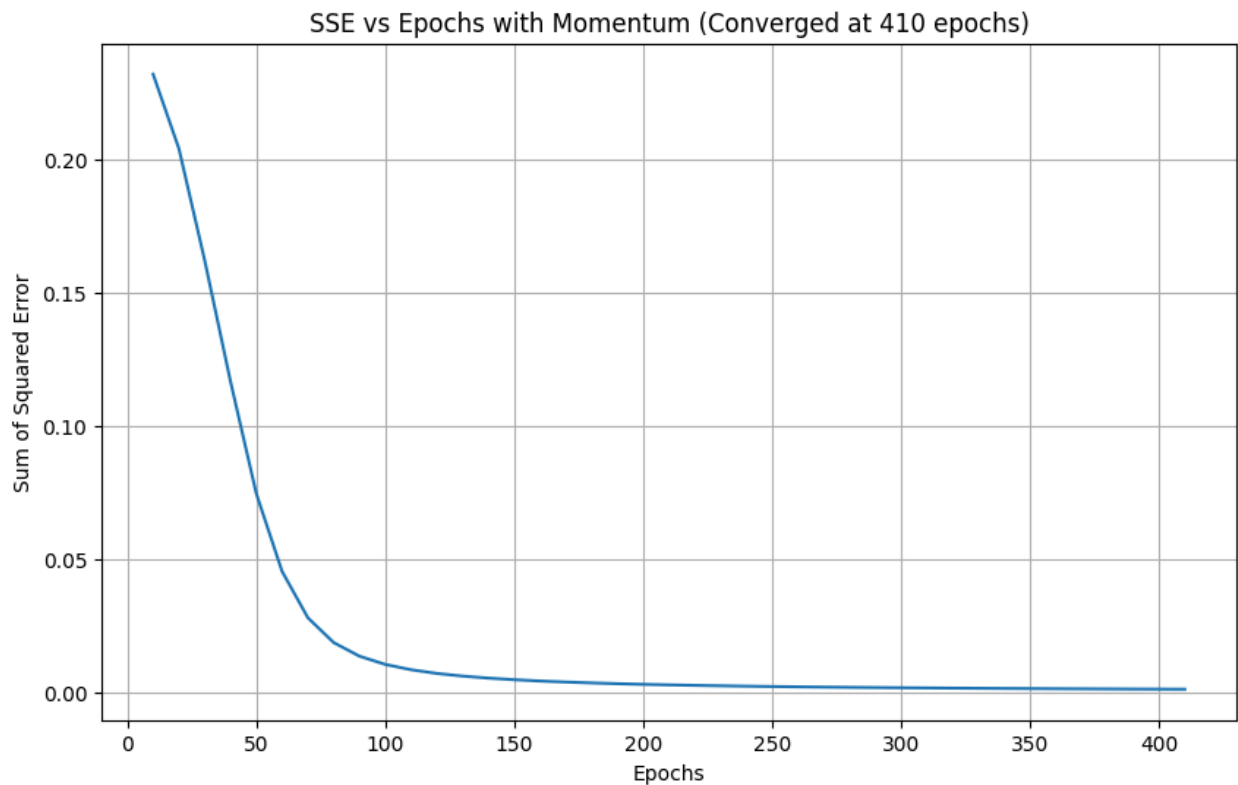
epochs_done += 10

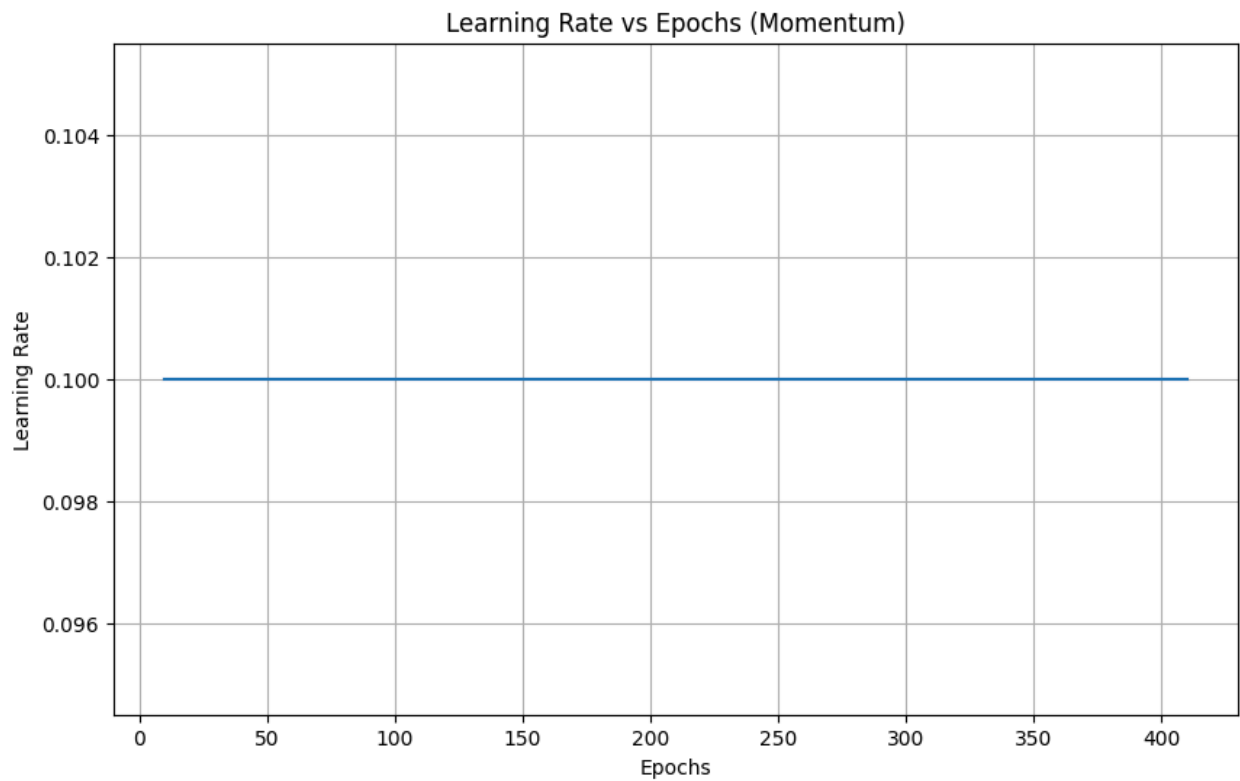
if loss < target_loss:
    converged = True

plt.figure(figsize=(10,6))
plt.plot(np.arange(10, epochs_done + 1, 10), losses_mom)
plt.title(f"SSE vs Epochs with Momentum (Converged at {epochs_done} epochs)")
plt.xlabel("Epochs")
plt.ylabel("Sum of Squared Error")
plt.grid(True)
plt.show()

plt.figure(figsize=(10,6))
plt.plot(np.arange(10, epochs_done + 1, 10), learning_rates_mom)
plt.title("Learning Rate vs Epochs (Momentum)")
plt.xlabel("Epochs")
plt.ylabel("Learning Rate")
plt.grid(True)
plt.show()

```





```
In [29]: initial_lr = 0.1
current_lr = initial_lr
losses_adapt = []
learning_rates_adapt = []
converged = False
epochs_done = 0
target_loss = 0.001
prev_loss = float('inf')

model = create_model(lr=current_lr)

while not converged:
    history = model.fit(X, y, epochs=5, verbose=0)
    loss = history.history['loss'][-1]
    losses_adapt.append(loss)
    epochs_done += 5

    if loss > 1.04 * prev_loss:
        current_lr *= 0.7
    elif loss < prev_loss:
        current_lr *= 1.05

    model.optimizer.learning_rate.assign(current_lr)
    learning_rates_adapt.extend([current_lr] * 5)
    prev_loss = loss

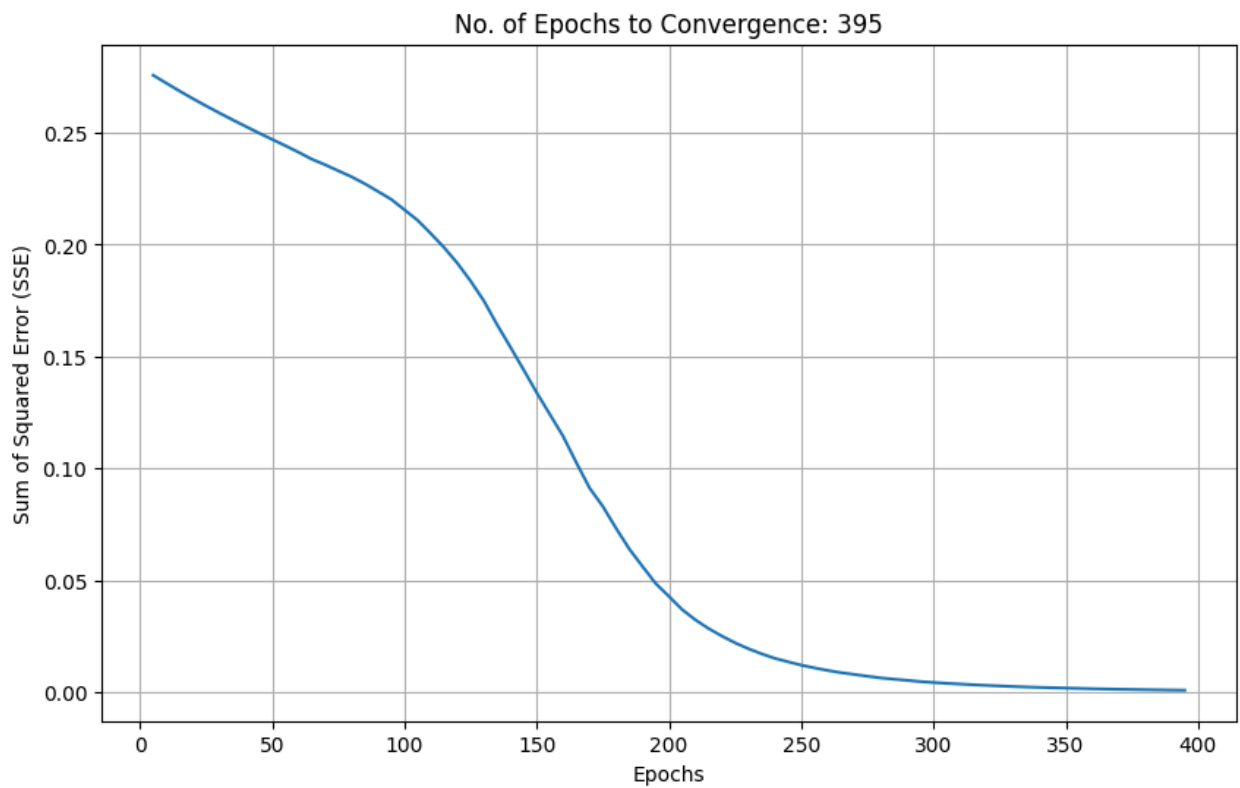
    if loss <= target_loss:
        converged = True
```

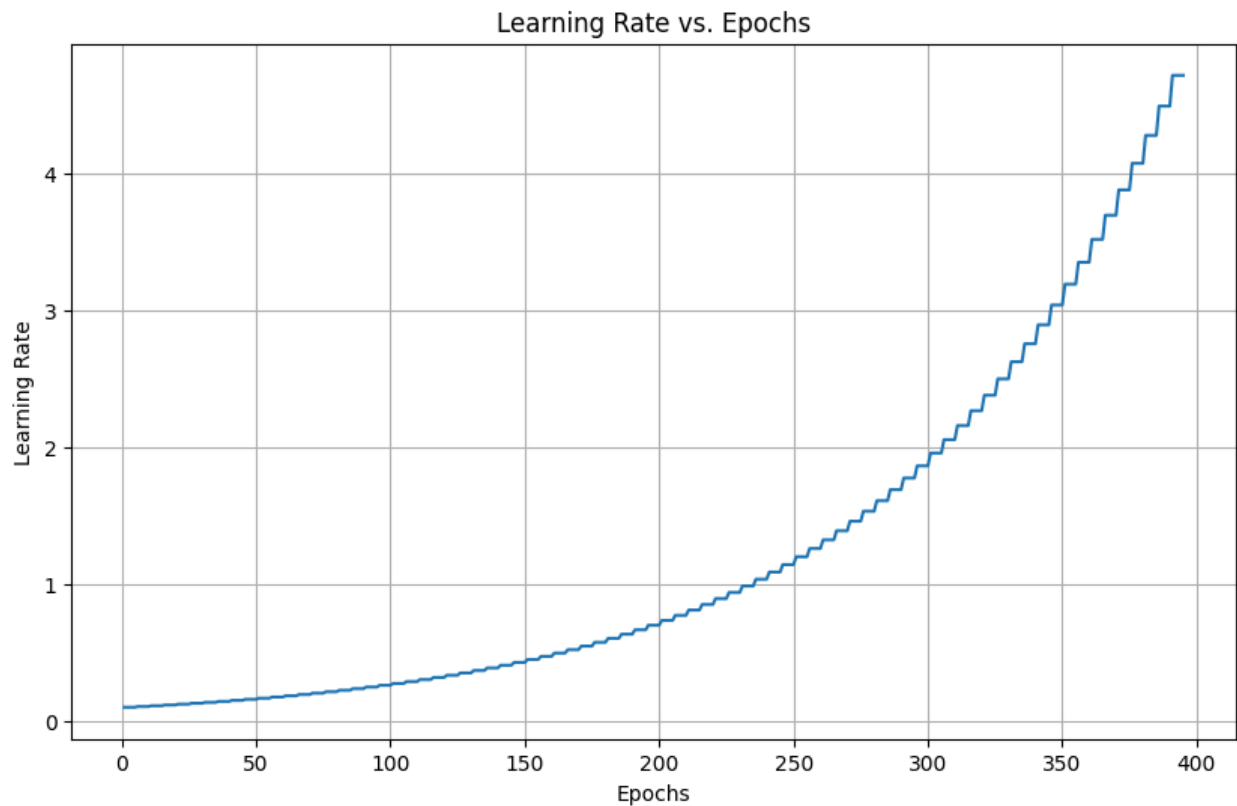
```

plt.figure(figsize=(10, 6))
plt.plot(np.arange(5, epochs_done + 1, 5), losses_adapt)
plt.title(f"No. of Epochs to Convergence: {epochs_done}")
plt.xlabel("Epochs")
plt.ylabel("Sum of Squared Error (SSE)")
plt.grid(True)
plt.show()

plt.figure(figsize=(10, 6))
plt.plot(range(1, epochs_done + 1), learning_rates_adapt)
plt.title("Learning Rate vs. Epochs")
plt.xlabel("Epochs")
plt.ylabel("Learning Rate")
plt.grid(True)
plt.show()

```





```
In [30]: initial_lr = 0.1
current_lr = initial_lr
momentum = 0.9
losses_adapt_mom = []
learning_rates_adapt_mom = []
converged = False
epochs_done = 0
target_loss = 0.001
prev_loss = float('inf')

model = create_model(lr=current_lr, momentum=momentum)

while not converged:
    history = model.fit(X, y, epochs=5, verbose=0)
    loss = history.history['loss'][-1]
    losses_adapt_mom.append(loss)
    epochs_done += 5

    if loss > 1.04 * prev_loss:
        current_lr *= 0.7
    elif loss < prev_loss:
        current_lr *= 1.05

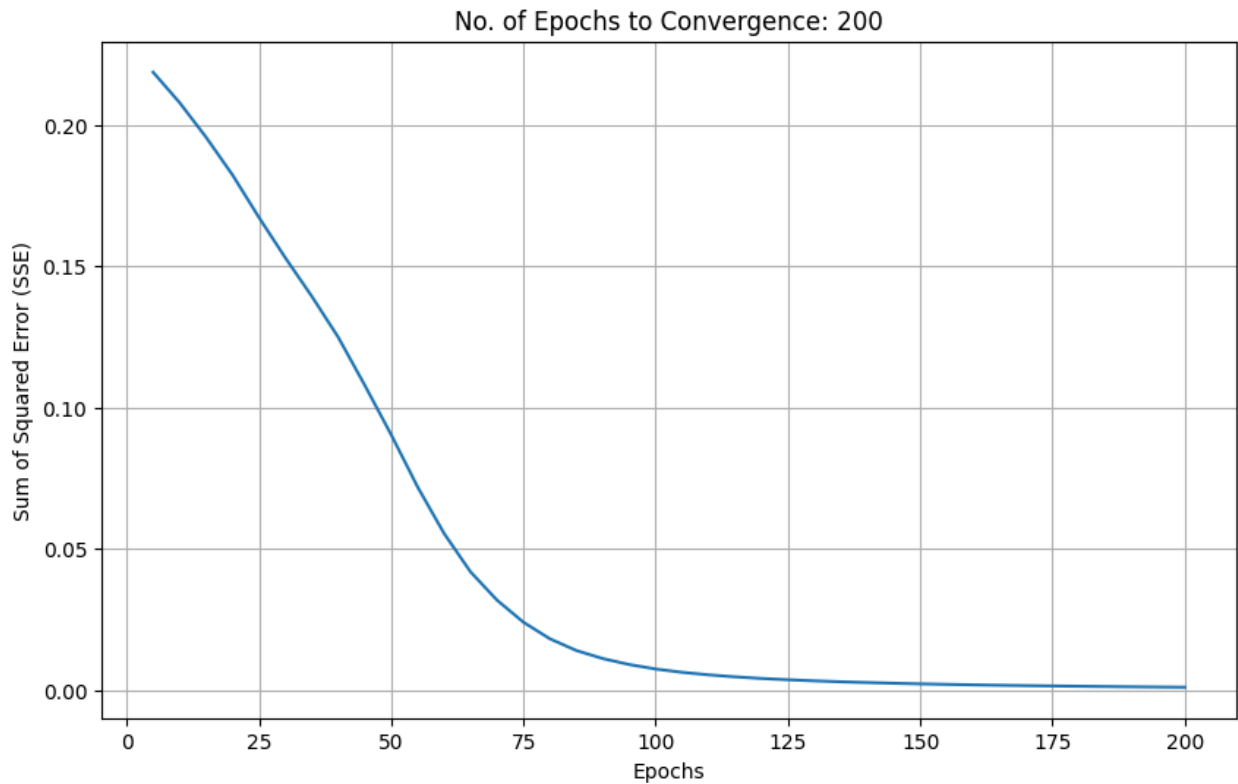
    model.optimizer.learning_rate.assign(current_lr)
    learning_rates_adapt_mom.extend([current_lr] * 5)
    prev_loss = loss

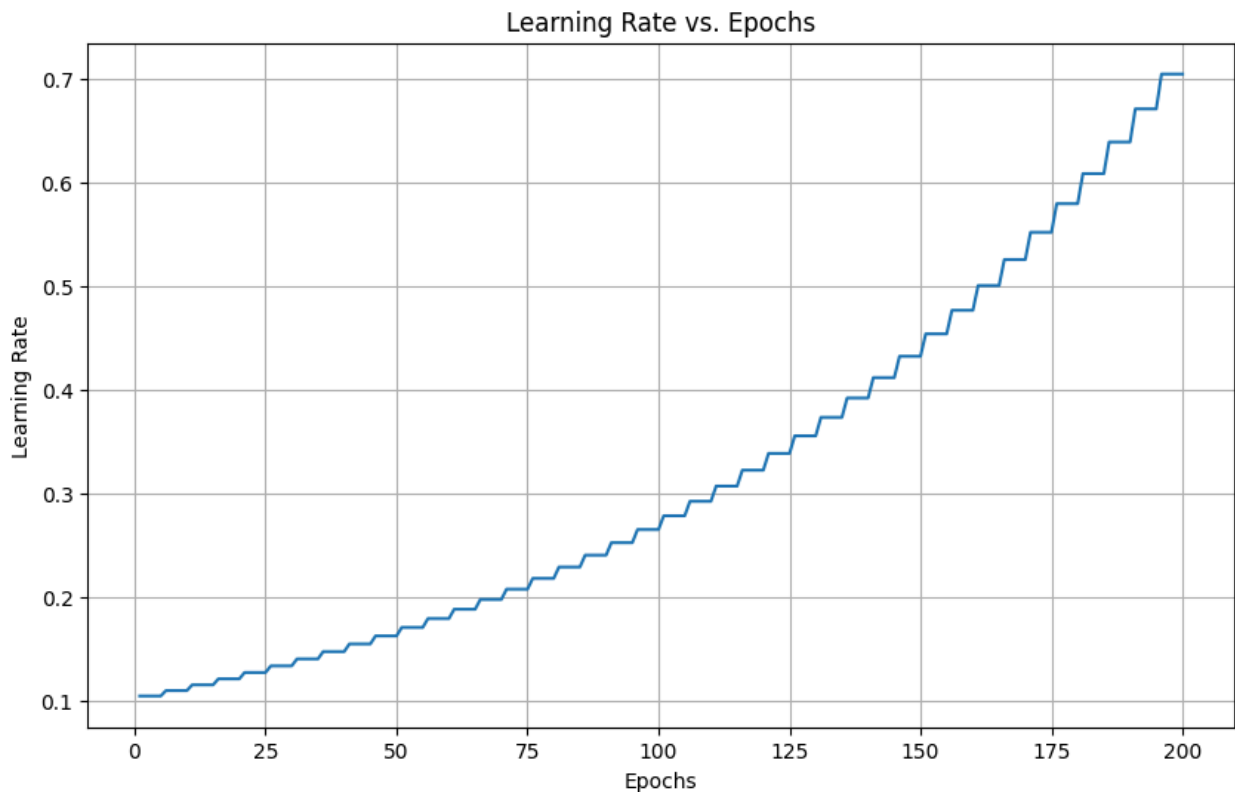
    if loss <= target_loss:
```

```
converged = True
```

```
plt.figure(figsize=(10, 6))
plt.plot(np.arange(5, epochs_done + 1, 5), losses_adapt_mom)
plt.title(f"No. of Epochs to Convergence: {epochs_done}")
plt.xlabel("Epochs")
plt.ylabel("Sum of Squared Error (SSE)")
plt.grid(True)
plt.show()

plt.figure(figsize=(10, 6))
plt.plot(range(1, epochs_done + 1), learning_rates_adapt_mom)
plt.title("Learning Rate vs. Epochs")
plt.xlabel("Epochs")
plt.ylabel("Learning Rate")
plt.grid(True)
plt.show()
```





1. How does a multilayer neural network solve the XOR problem?

Ans: A multilayer neural network solves the XOR problem by using hidden layers to learn non-linear decision boundaries. A single perceptron cannot solve XOR because it can only handle linearly separable data. The hidden layers transform the input into a space where the classes become separable. Using backpropagation, the network iteratively adjusts weights to minimize error and correctly model XOR.

2. In a backpropagation neural network, why are the middle layers called "hidden"?

Ans: The middle layers are called hidden layers because their outputs are not directly visible from the input or output of the network. They do not interact with the external environment. Instead, they perform internal computations and feature transformations. These layers help the network learn complex patterns between input and output.

3. How are hyperparameters different from model parameters?

Ans: Model parameters are the internal values of the network, such as weights and biases, learned during training. Hyperparameters are predefined settings that control the learning process. Examples include learning rate, number of epochs, and number of layers. Hyperparameters are set before training and are not

updated during training.

4. Explain the concept of accelerated learning in backpropagation.

Ans: Accelerated learning in backpropagation uses momentum to reduce oscillations and improve convergence speed during training. Momentum incorporates a fraction of the previous weight update into the current update. This helps the network move smoothly in the correct direction and avoid getting stuck in local minima. As a result, it speeds up convergence and stabilizes learning in noisy gradient regions.

5. What are two heuristics applied in adaptive learning rate?

Ans: Heuristic 1: If the change of the sum of squared errors has the same algebraic sign for several consequent epochs, then the learning rate parameter, α , should be increased.

Heuristic 2: If the algebraic sign of the change of the sum of squared errors alternates for several consequent epochs, then the learning rate parameter, α , should be decreased.